

Fruitfulness as Compression: Lean-Checked Theorems and a Measurement of All of Mathlib

Vasily Shablov*

October 2026

Draft

Abstract

We pose an open-ended question: what is the *fruitfulness* of a mathematical idea? We answer it with a definition: a named concept is a compression, and its value is what naming it saves. We then prove theorems about this definition in Lean 4 and measure it on all 771,129 constants of Mathlib.

Theorems.

- The saving from naming a body of size b used k times is exactly $(k - 1)(b - 1) - 1$.
- Unfolding all names at most doubles proof size per result, and the doubling is attained.
- With at most d citations per result, the sharp bound is the d -bonacci recurrence, also attained.

Measurements.

1. *Growth law.* Naively unfolded proof size grows by a factor of about 1.5 per dependency level ($R^2 = 0.98$), reaching about $10^{65.6}$ symbols.
2. *Rediscovery.* Ranking by reuse alone rediscovers the textbook core: `Set`, `Category`, `Functor`, `TopologicalSpace`, `Real`, `Module`, ...
3. *Two axes of taste.* Human curation of *concepts* is predicted by reuse (AUC 0.88). Human fame of *theorems* is predicted by depth (AUC 0.78) and not by reuse (AUC 0.54).
4. *A glue layer.* Instance declarations and structure projections are 12% of constants but receive 60% of all citations. Mathlib also obeys Zipf’s law of abbreviation. These are the same signatures as function words in language and currency metabolites in metabolism.

We correct an overreach in our own earlier statement (naive unfolding is not minimal lemma-free proof size). We connect the work to proof complexity, logical depth, compression progress, hierarchy in nature, and Wigner’s puzzle. We state nine open questions and five falsifiable conjectures.

1 Introduction

Where the question came from. The question started as a debate. One position holds that AI is good at well-posed yes/no problems but cannot handle open “why / what is” questions, because “there is nothing to optimize.” Rather than argue, we posed one such question and followed it:

What is the fruitfulness of a mathematical idea, and can it be measured?

*Formalization, measurement and drafting assisted by Claude (Anthropic).

Our answer. We take a compression view, in the tradition of minimum description length and of Whitehead’s remark that “civilization advances by extending the number of important operations which we can perform without thinking about them.”

- A definition or lemma is *named*: its body is written once and then cited.
- The value of a name is the description length it saves.
- This makes fruitfulness precise enough to prove theorems about, and concrete enough to measure on the largest library of formalized mathematics.

Contributions.

1. **Theorems, checked in Lean 4 with Mathlib** (Section 2; no `sorry`; standard axioms only).
 - The exact value of naming a concept.
 - A tight exponential bound on unfolding.
 - A sharp bound for bounded citation, via the d -bonacci recurrence.
2. **A complete measurement of Mathlib** (Sections 3–4).
 - A growth law for unfolded size.
 - Reuse rediscovers the textbook core.
 - Two distinct axes of human mathematical taste, each predicted by a computable measure.
 - A glue layer with the statistics of function words.
3. **Connections and open questions** (Section 5).
 - The deepest version of “are concepts necessary?” is the open Frege versus Extended Frege problem.
 - Human fame tracks Bennett’s logical depth; concept taste tracks Schmidhuber’s compression progress.
 - The same reuse statistics appear in language, metabolism and software.

2 Theory: what naming buys

Definition 1 (Library). *A library consists of:*

- a *size function* $\text{size} : \mathbb{N} \rightarrow \mathbb{N}$;
- *citation sets* $\text{deps}(i) \subseteq \{0, \dots, i-1\}$.

Result i has its own proof of size $\text{size}(i)$ and cites the results in $\text{deps}(i)$. Its unfolded size is

$$U(i) = \text{size}(i) + \sum_{j \in \text{deps}(i)} U(j),$$

which is the size of its proof if each cited result were re-proved in place.

Theorem 1 (Value of one name; `naming_saves`, `naming_pays_iff`). *Inlining a body of size b at k sites costs kb . Naming it costs $b + k$: one definition plus one reference per use. The saving is exactly*

$$kb - (b + k) = (k - 1)(b - 1) - 1.$$

- *Naming pays off if and only if $(k - 1)(b - 1) > 1$. In particular it always pays when $k \geq 2$ and $b \geq 3$ (`naming_pays_off`).*
- *A name used once never pays: it costs exactly one symbol (`naming_once`).*

The last clause is the *rule utility* constraint of the Sequitur grammar compressor [17], here derived rather than imposed.

Theorem 2 (Doubling; `Library.unfold_le_two_pow`, `completeLib_unfold`). If $\text{size} \leq S$, then $U(i) \leq S \cdot 2^i$, whatever the number of citations. Equality holds for the complete library, in which every result cites all earlier ones and has size 1: there $U(i) = 2^i$.

Theorem 3 (Sharp bound for bounded citation; `Library.unfold_le_tb`, `windowLib_unfold`). Let $T_d(i) = 1 + \sum_{j=\max(0, i-d)}^{i-1} T_d(j)$. If $\text{size} \leq S$ and $|\text{deps}(i)| \leq d$ for all i , then

$$U(i) \leq S \cdot T_d(i).$$

Equality holds for the library in which each result cites its d predecessors and has size 1.

The proof rests on an exchange lemma (`sum_le_sum_window`): among sets of at most d indices below i , a monotone function has its largest sum on the last d indices. Some consequences:

- $T_d(i)$ grows like ϕ_d^i , where ϕ_d is the d -bonacci constant, the root in $(1, 2)$ of $x^d = x^{d-1} + \dots + 1$.
- ϕ_2 is the golden ratio. The case $d = 2$ is formalized separately: $U(i) = F_{i+3} - 1 \geq 2^{\lfloor i/2 \rfloor}$ (`fibLib_unfold`, `fibLib_unfold_ge`).
- $\phi_d \rightarrow 2$ as $d \rightarrow \infty$, recovering the doubling theorem.

Every one of these exponential gaps is generated by a library of *linear* named size.

Remark (What these theorems do and do not say). U measures *copying*. It is not the size of the shortest lemma-free proof, and a clever re-prover can restructure a proof rather than inline it. For Frege systems, tree-like proofs polynomially simulate dag-like ones [13]. So, in that setting, reusing lemmas saves only polynomially, even though copying costs exponentially. The question of whether *new symbols* (definitions) can save super-polynomially is the Frege versus Extended Frege problem, which is open (Section 5). Our theorems are about naming versus copying, and they are tight for that.

3 Data and method

Export. We loaded all of Mathlib [15] (Lean v4.33.1), including private proof bodies, and recorded every constant: its kind, its module, the size of its body (number of distinct term nodes), and the constants cited in its body and statement. In a second pass, with environment extensions loaded, we recorded the size of its statement and whether it is an instance declaration or a structure projection.

Size of the data.

- 771,129 constants and 20.4 M citation edges.
- 279 cycle edges, from unsafe definitions, were dropped.
- The *visible* set has 315,085 constants: user-facing Mathlib theorems, definitions and inductive types, after removing compiler-generated auxiliaries.

Measures. For each constant c :

- *reuse* $k(c)$: the number of constants citing c ;
- *unfolded size* $U(c)$: as in Section 2, computed exactly in log-space;
- *height*: the longest citation chain below c ;
- *depth*: $\log_{10} U(c) - \log_{10}(\text{statement size})$, i.e. hidden work per symbol of statement.

Glue. We call instance declarations and structure projections *glue*, classified structurally rather than by name. They make up 12.3% of visible constants.

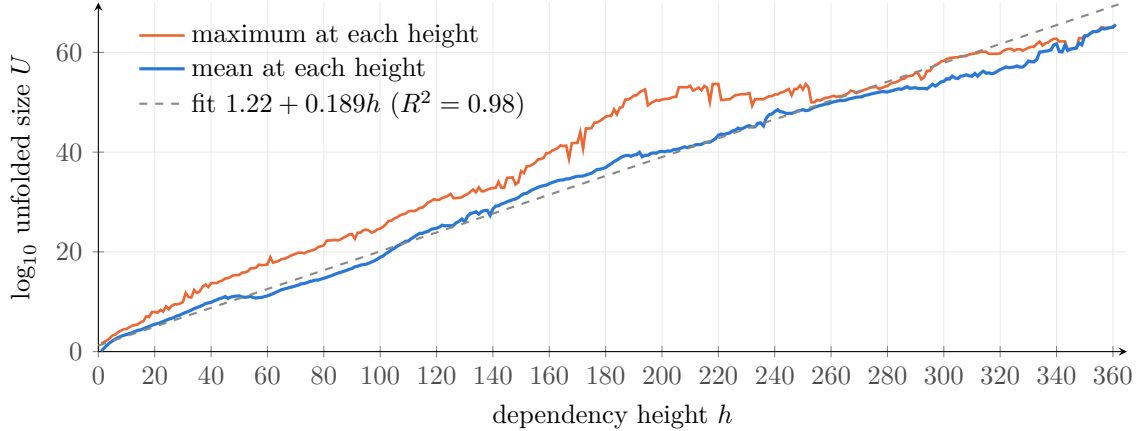


Figure 1: Naively unfolded size grows exponentially with dependency height, about $1.5\times$ per level, over 315,085 Mathlib constants.

Human labels. All labels come from Mathlib’s own human-curated files.

- *Famous*: Freek Wiedijk’s “100 theorems” and the “1000+ theorems” project, 294 declarations.
- *Curriculum*: the undergraduate curriculum list and the library overview, 686 declarations.

Statistics.

- AUC against all other non-glue constants of the same kind. 0.5 is chance and 1.0 is perfect.
- A *same-file* control: the probability that a labelled constant beats an unlabelled constant of the same kind in the same source file. This controls for “advanced files contain deeper things.”

4 Results

4.1 A growth law

Mean $\log_{10} U$ is almost exactly linear in height (Figure 1).

- **Fit.** Over all 315,085 constants, $\log_{10} U \approx 1.22 + 0.189 h$ with $R^2 = 0.979$, a factor of 1.55 per level. Fitting the per-height means gives 1.50 per level with $R^2 = 0.990$.
- **Stability across the range.** The three thirds of the height range give 1.45, 1.64 and 1.41.
- **Placement.** This is supercritical branching: well above the linear regime, and below the golden ratio, 1.618 per *result*, of the citing-two-predecessors library.
- **Scale.** The deepest result, `CompletelyPositiveMap.map_cstarMatrix_nonneg` at height 361, has $U \approx 10^{65.6}$.

4.2 Reuse rediscovers the textbook

With glue removed, the most-cited definitions and theorems are listed below. No notion of importance was supplied. The ranking is pure citation counting.

Human label	n	reuse k	unfolded U	depth	statement size
curriculum <i>definitions</i>	421	0.88 / 0.87	0.46 / 0.35	0.46 / 0.38	0.36 / 0.32
curriculum theorems	209	0.58 / 0.59	0.82 / 0.66	0.82 / 0.68	0.67 / 0.51
famous theorems	259	0.54 / 0.55	0.78 / 0.66	0.78 / 0.67	0.59 / 0.51

Table 1: Global AUC / same-file win rate, glue removed. 0.5 is chance.

Definitions (by citations)	Theorems (by citations)
Set, Category, Functor, TopologicalSpace, Real, Module, RingHom.id, CommRing, Finset, Semiring, RingHom, AddCommMonoid, AddCommGroup, Algebra, NormedAddCommGroup, LinearMap, Equiv, Opposite, ...	mul_one, add_zero, le_trans, zero_add, one_mul, Nat.cast_one, mul_comm, Finset.sum_congr, Set.ext, le_refl, mul_assoc, map_zero, le_antisymm, ...

The theorem column omits tactic-internal certificate lemmas (`Mathlib.Tactic.*`), which form a third, automation-generated kind of glue.

Observation. The top of the concept ranking is dominated by notions of *structure and sameness*: sets, maps, homomorphisms, equivalences, categories, functors, opposites. The single most-cited definition overall is `DFunLike.coe` (applying a structured map), at 115,373 citations. In a precise sense, the most compressive idea in modern mathematics is the morphism.

4.3 Two axes of human taste

Table 1 gives the main result.

- **Concepts.** Human-curated concepts are predicted by *reuse*: AUC 0.88, and 0.87 against peers in the same file. Depth does not predict them, and is even inverted (AUC 0.46): fundamental concepts are shallow.
- **Theorems.** Famous and curriculum theorems are predicted by *depth*: AUC 0.78–0.82, and 0.66–0.68 within the same file. Reuse barely predicts them (0.54–0.58).
- **Famous theorems are not reuse hubs.** 28.6% of famous declarations are never cited anywhere else in Mathlib, against 36.6% of all theorems, so they are only slightly more reused than average.

4.4 A glue layer, a heavy tail, and the law of abbreviation

Glue. Glue (instances and projections) is 12.3% of visible constants but receives 59.5% of all citations, including 77 of the 100 most-cited constants.

- Only 63 of the 904 human-labelled declarations are glue.
- This is the profile of *function words* in language: a small closed class carrying a large share of tokens and little content.
- It is also the profile of *currency metabolites* (ATP, NADH, water), which network biologists must remove before metabolic pathways become visible [14].

Heavy tail. Reuse is heavy-tailed (Figure 2).

- A discrete power-law maximum-likelihood fit [7] gives a density exponent $\alpha \approx 1.72$, 1.77 and 1.79 for $k \geq 10$, 30 and 100.

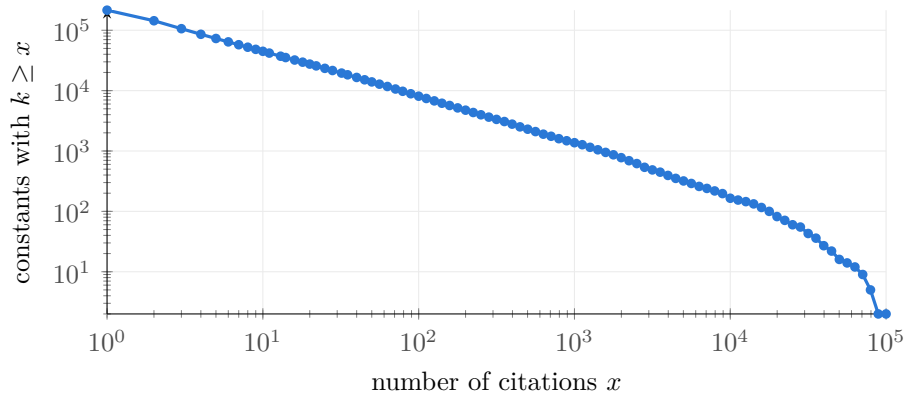


Figure 2: Reuse in Mathlib is heavy-tailed: 215,078 constants are cited at least once, 8,090 at least 100 times, and 165 at least 10,000 times.

- For comparison: word frequencies give $\alpha \approx 2$ (Zipf), and protein-domain family sizes and citation counts show similar tails [12, 18].

Law of abbreviation. In natural languages, frequent words are shorter (Zipf), a regularity proposed to be a universal consequence of compression [10]. Mathlib obeys it too.

- **Definitions.** Mean name length falls monotonically with reuse: 13.2 characters (never cited), 10.8 (cited 10–99 times), 8.9 (10^3 – 10^4), 8.2 (10^4 – 10^5), and 3 for **Set**.
- **Theorems.** Mean name length falls from 19.2 to 8.5.
- **Rank correlations.** Spearman $\rho(k, \text{name length}) = -0.16$ for definitions and -0.07 for theorems; $\rho(k, \text{body size}) = -0.22$ for definitions. Most constants have small k , so the global correlations are weak, but the trend across decades of k is monotone.

Mathlib’s names are derived from statements by convention, so this is not designed; it emerges from which concepts get reused.

5 Connections and open questions

5.1 Are concepts necessary? Proof complexity

Naming comes in two strengths.

- **Lemmas** reuse proved statements: dag-like versus tree-like proofs.
 - The gap is exponential for resolution [1].
 - It is polynomial for Frege [13].
- **Definitions** introduce new symbols. This is Tseitin’s extension rule; Frege plus extension is Extended Frege [8].
 - Whether Extended Frege is super-polynomially stronger than Frege is a central open problem.

Higher up, the answer is known to be dramatic. Gödel [11] showed that passing to higher-order logic shortens some proofs by more than any computable function, and Boolos’s “curious inference” [3] gives a concrete first-order-valid argument whose first-order proofs are physically unwritable but which has a short second-order proof.

Question 1 (Are new concepts forced?). Are *definitions* ever super-polynomially necessary beyond lemma reuse? This is Frege versus Extended Frege, restated as the question “are human concepts forced or merely convenient?”

Question 2 (Empirical proof complexity). Mathlib is a vast sample of definitions in actual use. Can it locate *where* definitions give large savings, and so supply empirical evidence on a problem that has resisted proof?

5.2 Value has two axes: depth and leverage

The split in Table 1 matches two independent theories of value.

- **Bennett’s logical depth** [2]. An object is valuable when it is the product of a long computation that cannot be shortcut. Famous theorems are deep: a short statement certifies a large amount of hidden work.
- **Schmidhuber’s compression progress** [19]. Interest equals improvement of the observer’s compressor. Our $(k - 1)(b - 1) - 1$ is exactly the compression progress of one new concept, and reuse predicts concept taste.

We know of no previous large-scale test of either thesis against human judgments of mathematical value, though a fuller literature search is warranted.

Question 3 (The residual). Are depth and leverage the only two axes of mathematical value? What explains the remaining 12–22% of AUC: surprise, beauty, *bridging* between distant areas, or historical accident?

Question 4 (Computability of taste). Ideal fruitfulness, meaning the optimal library of concepts for all of mathematics, is a minimal-grammar problem. It is NP-hard even for finite corpora [5], and in the limit it is Kolmogorov complexity, which is uncomputable. So no agent, human or machine, can optimize fruitfulness exactly; all can only approximate it from below. Is the human advantage in choosing directions, if any, then a matter of better approximation rather than a different kind of faculty?

5.3 A glue law across nature

Function words, currency metabolites, tiny utility packages (the `left-pad` incident), and Mathlib’s instances share a profile: few units, an enormous share of uses, little content, and invisible to anyone asked what matters.

Question 5 (The glue theorem). Must every system that grows by reuse under a description-length cost develop (i) heavy-tailed reuse, (ii) a high-frequency, low-content glue layer, and (iii) abbreviation of the frequent? Preferential attachment [18] explains (i), and compression arguments [10] explain (iii). Is there a single derivation of all three?

Mathlib is a natural *model organism* for such laws.

- In biology, edges are inferred and noisy. In language, “concept” is fuzzy.
- In Mathlib, every edge is exact and every unit is machine-checked.

Question 6 (Mathematics as an evolving genome). Protein-domain statistics are explained by birth–death–innovation models [12]. Does the same model, fitted to Mathlib’s history (generalization as duplication, imported concepts as horizontal transfer), reproduce its reuse distribution?

5.4 Hierarchy, decoupling, and the effectiveness of mathematics

Simon’s watchmakers. Simon’s parable [20] contrasts a watchmaker who builds from stable sub-assemblies with one who does not. The first has an exponential advantage under interruption, which Simon offered as the reason nature is hierarchical. Our theorems are the corresponding statement about *size*.

Proof irrelevance is scale separation. In Lean, any two proofs of the same proposition are equal. Later results therefore depend only on *statements*, never on proofs. This is the logical form of the physicist’s decoupling of scales: chemistry needs the periodic table, not quantum chromodynamics. The depth $\log U - \log(\text{statement})$ measures how much micro-detail a statement forgets.

Question 7 (Wigner’s puzzle as compression). Is the “unreasonable effectiveness” of mathematics in physics [22] a consequence of both being hierarchical compressions under scale separation? Formalized physics libraries now exist. Do they reuse the same top concepts as Mathlib, with the same tail exponent and glue layer?

5.5 Delayed reward and understanding

Question 8 (How delayed is the reward?). The strongest form of the “nothing to optimize” objection is that fruitfulness is visible only decades later; Lobachevsky is the canonical example. Mathlib’s version history records when each declaration and each citation appeared. Does early reuse, after months, predict late reuse, after years? If it does, the reward horizon is short and learnable. If it does not, the objection gains real empirical support.

Question 9 (Understanding as conditional compression). Thurston held that mathematics is about human understanding [21]. Define: an agent *understands* a proof when its size, relative to the agent’s library of named concepts, fits within the agent’s working capacity. This is conditional description length with a library in place of a universal machine, consistent with chunking in cognition [6, 16]. On this definition, “can humans understand machine mathematics?” becomes the quantitative question “can the machine’s library be taught?”

5.6 Falsifiable conjectures

Conjecture 1 (Growth law). *In future versions of Mathlib, the per-level unfolding factor stays in $[1.4, 1.7]$ with $R^2 > 0.95$.*

Conjecture 2 (Two axes). *In other large formal libraries (Isabelle AFP, Rocq/MathComp), human-curated concepts are predicted by reuse and famous theorems by depth, each with $AUC > 0.75$. Reuse does not predict fame ($AUC < 0.6$).*

Conjecture 3 (Glue share). *In any mature formal library with typeclass-style inference, a structurally defined glue class of at most 15% of constants receives a majority of citations.*

Conjecture 4 (Concepts are rediscovered). *Strip names from Mathlib’s proof terms and run compression-driven library learning [9, 4]. The highest-value recovered abstractions substantially overlap the top concepts of Section 4.*

Conjecture 5 (Morphisms are the universal compressor). *In every sufficiently large formal library, the most-cited definitions are dominated by notions of map and sameness (morphism, homomorphism, equivalence, functor) rather than by specific objects.*

6 Limitations

- **The labels are biased.** They record what formalizers chose to curate.
- **U is a copying measure.** It counts each citation once per citing result. It is not an occurrence count inside proof terms, and not a lower bound on proof size.

- **The glue classification is incomplete.** It is structural but leaves out tactic-certificate lemmas, which we report separately.
- **One snapshot.** All measurements come from a single version of Mathlib. Temporal claims are stated as conjectures.
- **The reported correlations are observational.**

7 Conclusion

An open “what is” question led us to:

- a definition;
- theorems proved in Lean, including a sharp one;
- a growth law over 315,085 objects;
- a mechanical measure that recovers human concept taste at AUC 0.88 and theorem taste at AUC 0.78;
- connections to proof complexity, information theory, biology, linguistics and physics;
- questions we cannot yet answer.

Fruitfulness is not unmeasurable. It is delayed and, in its ideal form, uncomputable, for every kind of mind.

A Lean index

All results are in `Fruitfulness.lean`, namespace `Fruitfulness`. They use only `propext`, `Classical.choice` and `Quot.sound`, with no `sorry`.

Statement	Lean name
Saving $(k-1)(b-1)-1$	<code>naming_saves</code>
Pays iff $(k-1)(b-1) > 1$; pays when $k \geq 2, b \geq 3$	<code>naming_pays_iff</code> , <code>naming_pays_of</code>
Used once never pays	<code>naming_once</code>
Unfolded size $\geq$ own size	<code>Library.size_le_unfold</code>
$U(i) \leq S(d+1)^{i+1}$	<code>Library.unfold_le</code>
$U(i) \leq S 2^i$ (any d)	<code>Library.unfold_le_two_pow</code>
Complete library: $U(i) = 2^i$	<code>completeLib_unfold</code>
Exchange lemma	<code>sum_le_sum_window</code>
T_d is monotone	<code>tb_mono</code>
Sharp bound $U(i) \leq S T_d(i)$	<code>Library.unfold_le_tb</code>
Attained by the window library	<code>windowLib_unfold</code>
Fibonacci library: $U(i) = F_{i+3} - 1 \geq 2^{\lfloor i/2 \rfloor}$	<code>fibLib_unfold</code> , <code>fibLib_unfold_ge</code>

Reproduction: `Export.lean` and `Flags.lean` export the graph; `analyze.py`, `robustness.py` and `paper_stats.py` compute every number in this paper.

References

- [1] E. Ben-Sasson, R. Impagliazzo, A. Wigderson. Near optimal separation of tree-like and general resolution. *Combinatorica* 24 (2004).
- [2] C. H. Bennett. Logical depth and physical complexity. In *The Universal Turing Machine: A Half-Century Survey*, 1988.

- [3] G. Boolos. A curious inference. *J. Philosophical Logic* 16 (1987).
- [4] M. Bowers et al. Top-down synthesis for library learning. *POPL* 2023.
- [5] M. Charikar et al. The smallest grammar problem. *IEEE Trans. Inf. Theory* 51 (2005).
- [6] W. G. Chase, H. A. Simon. Perception in chess. *Cognitive Psychology* 4 (1973).
- [7] A. Clauset, C. R. Shalizi, M. E. J. Newman. Power-law distributions in empirical data. *SIAM Review* 51 (2009).
- [8] S. A. Cook, R. A. Reckhow. The relative efficiency of propositional proof systems. *J. Symbolic Logic* 44 (1979).
- [9] K. Ellis et al. DreamCoder: bootstrapping inductive program synthesis with wake-sleep library learning. *PLDI* 2021.
- [10] R. Ferrer-i-Cancho et al. Compression as a universal principle of animal behavior. *Cognitive Science* 37 (2013).
- [11] K. Gödel. Über die Länge von Beweisen. *Ergebnisse eines math. Kolloquiums* 7 (1936).
- [12] E. V. Koonin, Y. I. Wolf, G. P. Karev. The structure of the protein universe and genome evolution. *Nature* 420 (2002).
- [13] J. Krajíček. *Bounded Arithmetic, Propositional Logic, and Complexity Theory*. Cambridge University Press, 1995.
- [14] H. Ma, A.-P. Zeng. Reconstruction of metabolic networks from genome data and analysis of their global structure. *Bioinformatics* 19 (2003).
- [15] The mathlib Community. The Lean mathematical library. *CPP* 2020.
- [16] G. A. Miller. The magical number seven, plus or minus two. *Psychological Review* 63 (1956).
- [17] C. G. Nevill-Manning, I. H. Witten. Identifying hierarchical structure in sequences: a linear-time algorithm. *JAIR* 7 (1997).
- [18] D. de Solla Price. A general theory of bibliometric and other cumulative advantage processes. *JASIS* 27 (1976).
- [19] J. Schmidhuber. Driven by compression progress. In *Anticipatory Behavior in Adaptive Learning Systems*, Springer, 2009.
- [20] H. A. Simon. The architecture of complexity. *Proc. Amer. Phil. Soc.* 106 (1962).
- [21] W. P. Thurston. On proof and progress in mathematics. *Bull. AMS* 30 (1994).
- [22] E. Wigner. The unreasonable effectiveness of mathematics in the natural sciences. *Comm. Pure Appl. Math.* 13 (1960).